

Práctica Nro 4 - Árboles Introducción

1 - Revisión Conceptual.

Nota: Se sugiere antes de comenzar con la resolución de los problemas, avanzar con la revisión conceptual.

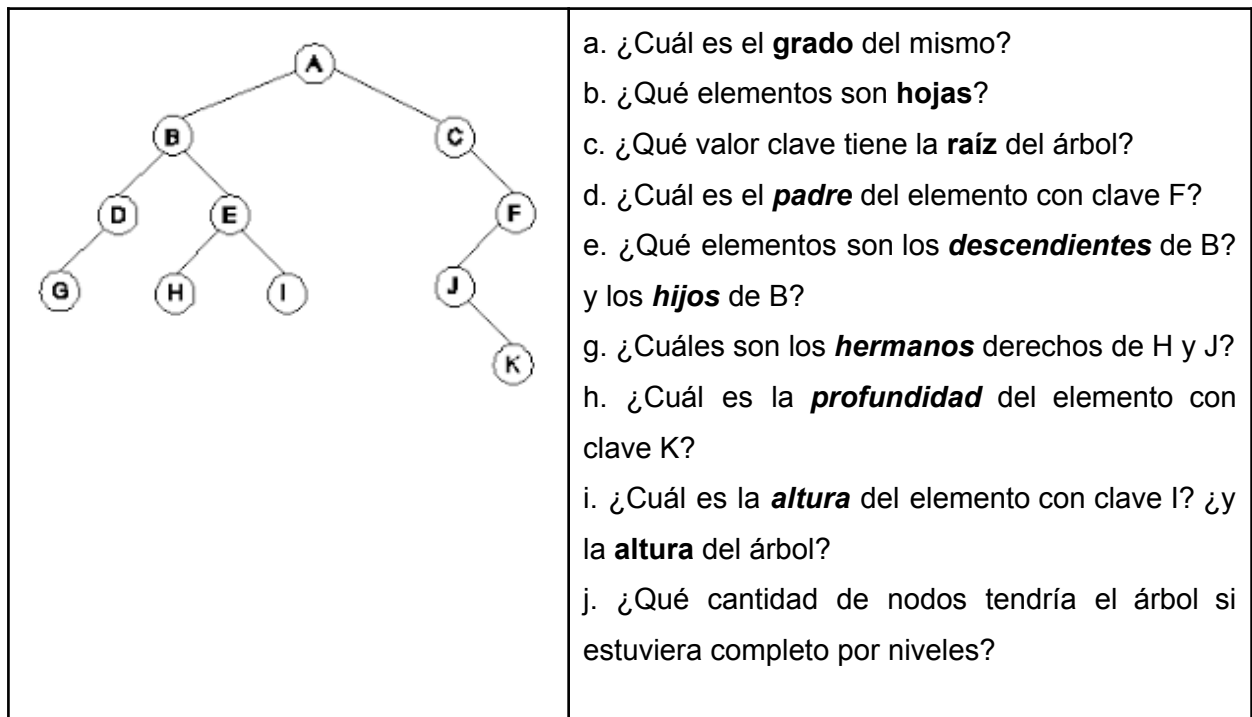
Analice los materiales de teoría sobre árboles y responda a las siguientes preguntas:

1- ¿Cuáles son las características de la estructura de datos árbol?

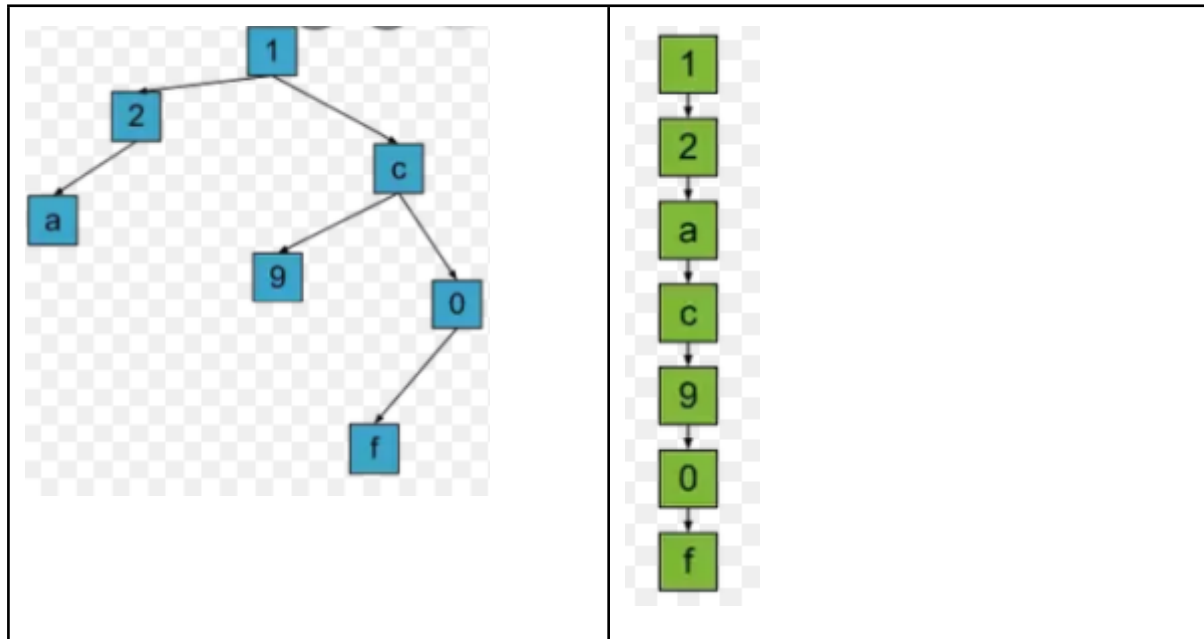
2- ¿Cómo se denomina a un árbol que tiene grado 2?

3- ¿Qué entiende por el término **clave** de un elemento del árbol?

4 - Dado el siguiente Árbol:



5 - Se presenta la misma información (caracteres) contenida en un árbol binario y en una lista enlazada.



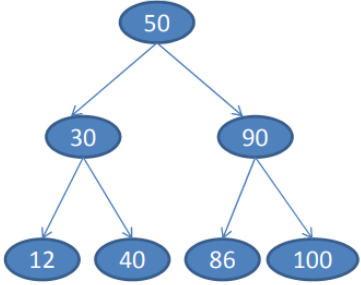
5.1 - ¿Qué estrategia usaría en cada caso para determinar si el valor 'f' está contenido en la estructura de datos? ¿y el valor '7'?

6 - Realice una comparación entre **listas enlazadas**, **arreglos** y **árboles** teniendo en cuenta los criterios: contenido (homogéneo/heterogéneo), acceso a los elementos (directo, lineal, no lineal), manejo de la memoria (estática/dinámica).

	Listas Enlazadas	Arreglos	Árboles
Contenido			
Acceso a los elementos			
Manejo de la memoria			

2 - Problemas

1 - Dado el árbol binario que se presenta en la parte izquierda de la tabla, determine a qué tipo de recorridos pertenecen los resultados que se presentan en la columna de la derecha.

	a) 12, 30, 40, 50, 86, 90, 100 b) 50, 30, 12, 40, 90, 86, 100 c) 12, 40, 30, 86, 100, 90, 50
---	---

Pregunta: Si te pidieran que dibujaras un árbol binario a partir de contar con los recorridos en pre-orden y post-orden. Consideras que habría suficiente información para ello. Tengamos en cuenta que las claves de los elementos del árbol serán únicas.

Nota: piensa en el siguiente ejemplo: resultado pre-orden ABC, resultado post-orden CBA ... puedes construir el árbol original a partir de esta información?

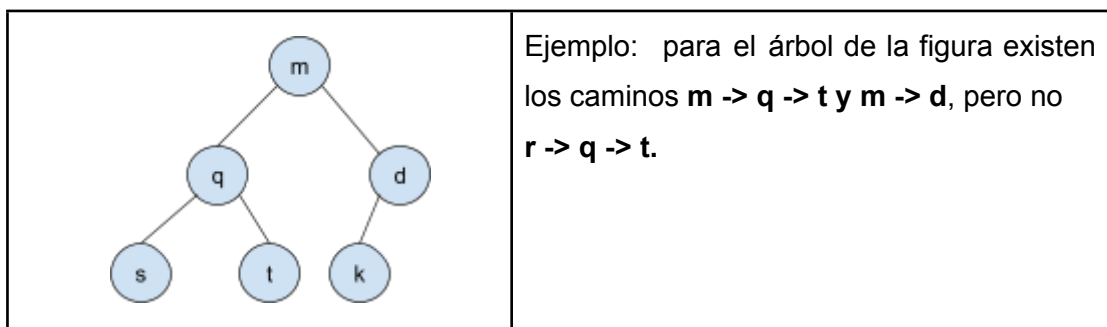
2 - Dada la siguiente definición de la estructura de un árbol binario de enteros usando punteros:

```
typedef struct nodoA {  
    int dato;  
    struct nodoA* hizq;  
    struct nodoA* hder;  
}*Árbol;
```

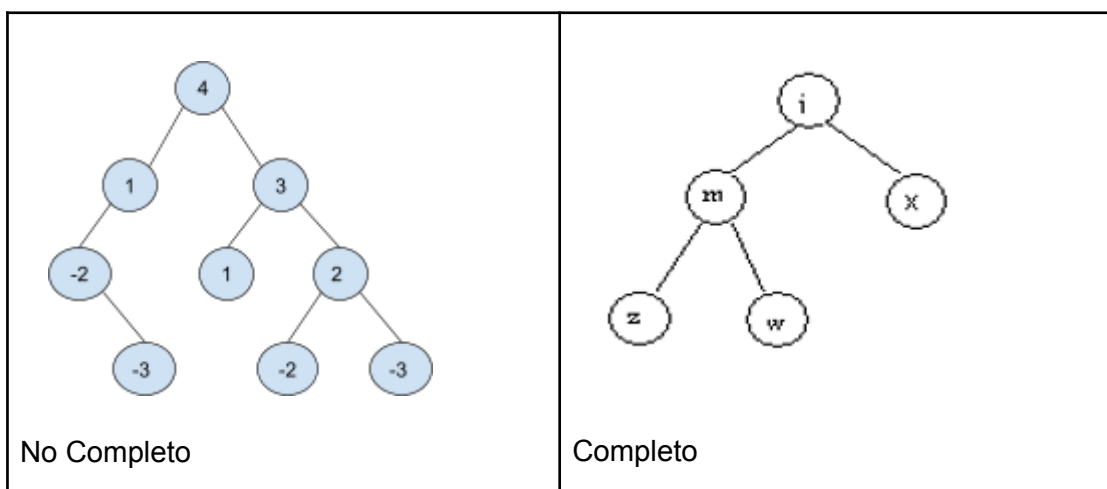
Nota: AB significa árbol binario

1. Diseñe e implemente el algoritmo **estaVacio**(Árbol A) que recibe un árbol binario y retorna 1 en caso que esté vacío y 0 en otro caso.
2. Implemente un algoritmo denominado **esHoja**, que determine si un elemento de un árbol binario es hoja . Definición: int **esHoja**(Árbol A).
3. Implemente un algoritmo denominado **esPadre**, que determine si el elemento que recibe como parámetro tiene descendientes. Definición: int **esPadre**(Árbol A).
4. Utilizando el algoritmo del punto 3, implemente una solución recursiva que permita contar las hojas de un árbol binario. Definición: int **cantHojas**(Árbol A);

5. Utilizando el algoritmo punto 4, implemente una solución recursiva que permita contar los nodos que son padres y que tengan los dos hijos presentes. Definición: **int cantPadres(Árbol A)**.
6. Cambiaría la solución si se desea imprimir las hojas de un árbol binario cuyo valor se encuentra comprendido en rango dado. void **imprimirHojas(Árbol A, int a, int b)**;
7. Dado un árbol binario y un valor X, determine la cantidad de claves con valor X que existen en el mismo **int cantClaves(Árbol A, int X)**.
8. Diseñe e implemente en C un módulo llamado **igualesAB**, que determine si dos árboles binarios que almacenan números enteros son iguales, es decir, si tienen la misma forma y contenido. Realice una versión recursiva e iterativa. Defina los tipos necesarios para resolver el problema.
9. Diseñe e implemente en C un módulo que, dados un árbol binario y un camino contenido en una lista de elementos, determine si existe ese camino en el árbol, teniendo en cuenta que el camino debe comenzar necesariamente en la raíz.



10. Un árbol binario AB se dice completo si todos sus nodos tienen dos descendientes, excepto las hojas. Diseñe e implemente en C una función que dado un árbol binario retorne verdadero si el árbol es completo y falso en otro caso.



11. Analice la siguiente función y explique qué se busca resolver a través de la misma.

¿Requiere pre-condiciones?

```
int misterio(Árbol raiz) {  
    if (raiz->hder == NULL)  
        return(raiz->dato);  
    else  
        return(misterio(raiz->hder));  
}
```